enormous; in fact, it's pretty tiny by modern standards. The HTTP specification itself does not place an upper limit on header sizes, but web servers tend to. For example, Apache is normally configured at 8 KB – that's approximately 8100 ASCII characters, which isn't much, especially when you consider that this limit applies to the entire set of headers plus the request line. The other data-passing mechanisms of HTTP do not fare any better. There are size limits on URLs and POST data… and imagine the mess you'd create constructing and managing URLs with thousands of characters in their query strings… Ugh.

So, we have a problem to solve if we want to accumulate user data to improve our application. Sending data from the client side has some tight restrictions in terms of how much of it can be sent in a single HTTP request. In today's world an 8KB budget for header data is quickly eroded, and this could in fact be a lot lower when factoring in, say, an nginx proxy configured for only 4KB of headers. Storing data server-side therefore offers some obvious advantages, and there are multiple ways to approach this, but the one we're going to focus on in this section is PHP's built-in session support.

A Basic PHP Session

Put simply, PHP sessions are data that is stored between requests and exposed in the `$_SESSION` superglobal. Users are assigned a unique ID, which is usually stored in a `PHPSESSID` cookie (it can also be inserted into the URL if required), and PHP uses it to store and retrieve data. The cookie name is configurable with the `session.name` ini directive, so if you prefer another name, simply set `session.name = MYSESSIONID` in your ini file. The storage mechanism is the filesystem by default, though other storage "modules" are available: A database is an obvious alternative, and two popular in-memory solutions are

- Redis https://github.com/phpredis/phpredis?tab=readme-ov-file#php-session-handler
- Memcached https://www.php.net/manual/en/memcached.sessions.php

We will focus on filesystem-based sessions in this section. To start a session, you call the built-in `session_start()` function (PHP can also be configured to always activate a session with the `session.auto_start = 1` ini directive). The official PHP documentation states that "a record of a session is not created" until data is added to the `$_SESSION` superglobal, but certainly with PHP 8.4 this does not seem to be the case: